

NOTEBOOK

Video-01 (Topic: The Multimeter)

- Aged and Modern electronics

↳ built on V, I, R

↓ voltage ↓ Current ↳ Resistance

described in "Ohm's law"

$$R = V/I, \quad V = RI, \quad I = V/R$$

⊗ Multimeter:

- Capable of measuring all those important parameters (V, R, I)
- easy to use • low cost

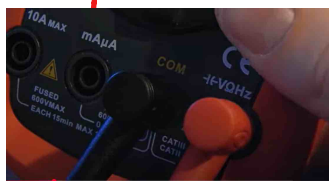
use:

- select ohms sign
- stick probe in the right socket.



(Resistance measure)

↳ black probe → connect to common socket.
↳ Red probe → must only be changed if measure current voltage & resistance is always in same socket



- connect the probes to the both side of resistance



N.B: Measuring resistance in the built ckt will fail cause current choose the way of least resistance.

• sign mean Ω sign



- called 'continuity'
- meter will beep when $R = 0 \Omega$ (Almost)

betⁿ two probes $\rightarrow$ great way to check cable break

$\rightarrow$ When cable break "no beep".



(Voltage measure) [sign: 'V=='; 'V~']

- to measure $\rightarrow$ connect $\rightarrow$ Red probe to +ve side
 $\rightarrow$ in parallel connection $\rightarrow$ Black " " -ve side $\rightarrow$ of power source

(Current measure) $\rightarrow$ 'HARDEST PART'!!

- Switch Red probe to 10A Max socket.

$\rightarrow$ why? to measure higher current.

For measurement $\rightarrow$

- Firstly, Open the ckt
- Connect probe to the opening point of the ckt.

if we measure in small range then the fuse will be down for further measurement. That's why, then unscrew $\rightarrow$ locate the smaller fuse and replace it with same values.

Video-2: Controlling LED Brightness using PWM

* Controlling LED Brightness can easily be achieved by using

PWM (Pulse Width Modulation)

→ allows for efficient dimming without burning out the LEDs

→ How? using microcontroller: **Arduino**

* Powering the LEDs (Using Power supply): Lowering the voltage beneath the forward voltage level reduces current consumption.

→ prevent from LEDs burnout
→ extends the lifespan.

* Powering the LEDs

(i) Using potentiometer: (Use to fix voltage supply on LEDs)

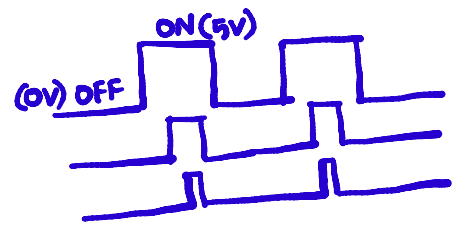
* It is an easiest way and fast solution.

* But it is insufficient for High power LEDs

→ i) waste energy
→ ii) require expensive compnts.

(ii) Using PWM:

→ operates by rapidly switching on/off
→ creating a perceived dimming effect while maintaining constant voltage. → control over brightness.
→ the LED is fully bright with 100% duty cycle.



How to set PWM?

```
int ledpin=3;
int potentiometer=A0;
int potentiometervalue;

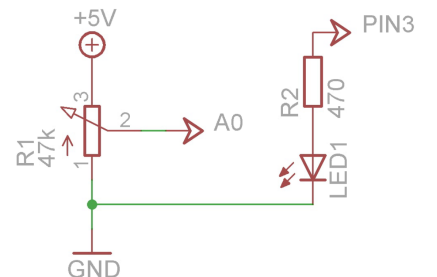
void setup() {
  pinMode(ledpin, OUTPUT);
  pinMode(potentiometervalue, INPUT);
}

void loop() {
  potentiometervalue=analogRead(potentiometer);
  potentiometervalue=map(potentiometervalue, 0, 1023, 0, 255);
  analogWrite(ledpin, potentiometervalue);
}
```

* AnalogWrite() function can generate PWM signal.

* signals feed betⁿ 0-255 values
(0V) (5V)

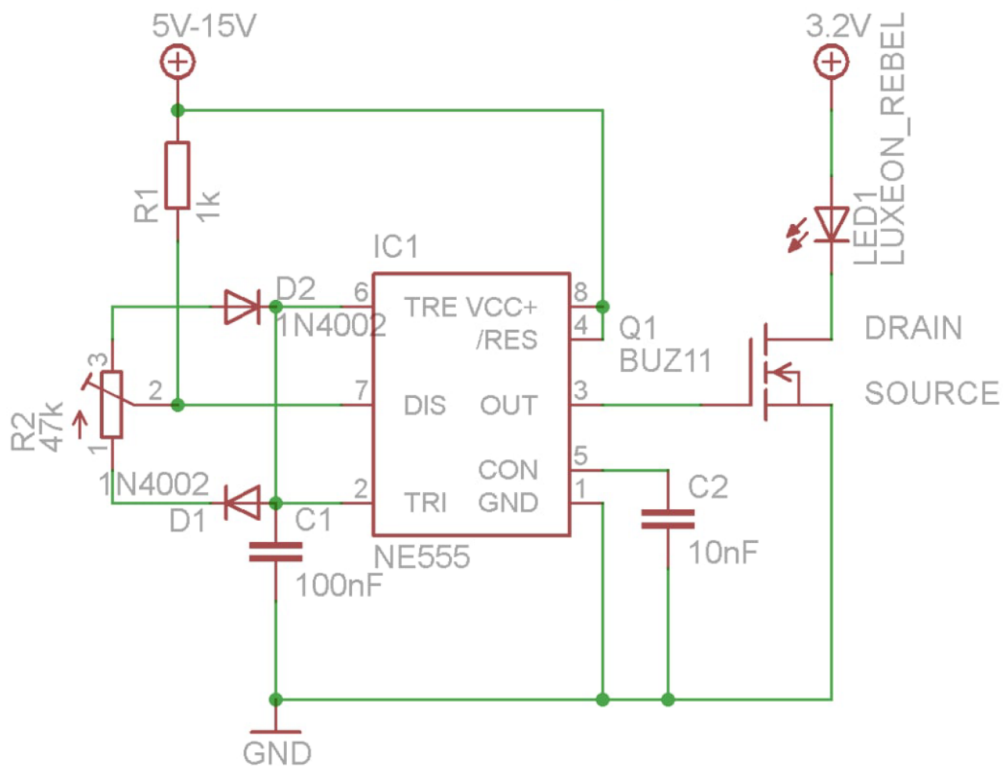
* The arduino's analog write function allow users to create PWM signals by varying voltage levels, demonstrating how to manipulate LED Brightness effectively. A potentiometer is used to adjust this value.



<http://www.youtube.com/user/greatscottlab>

* Using 555 timer chip

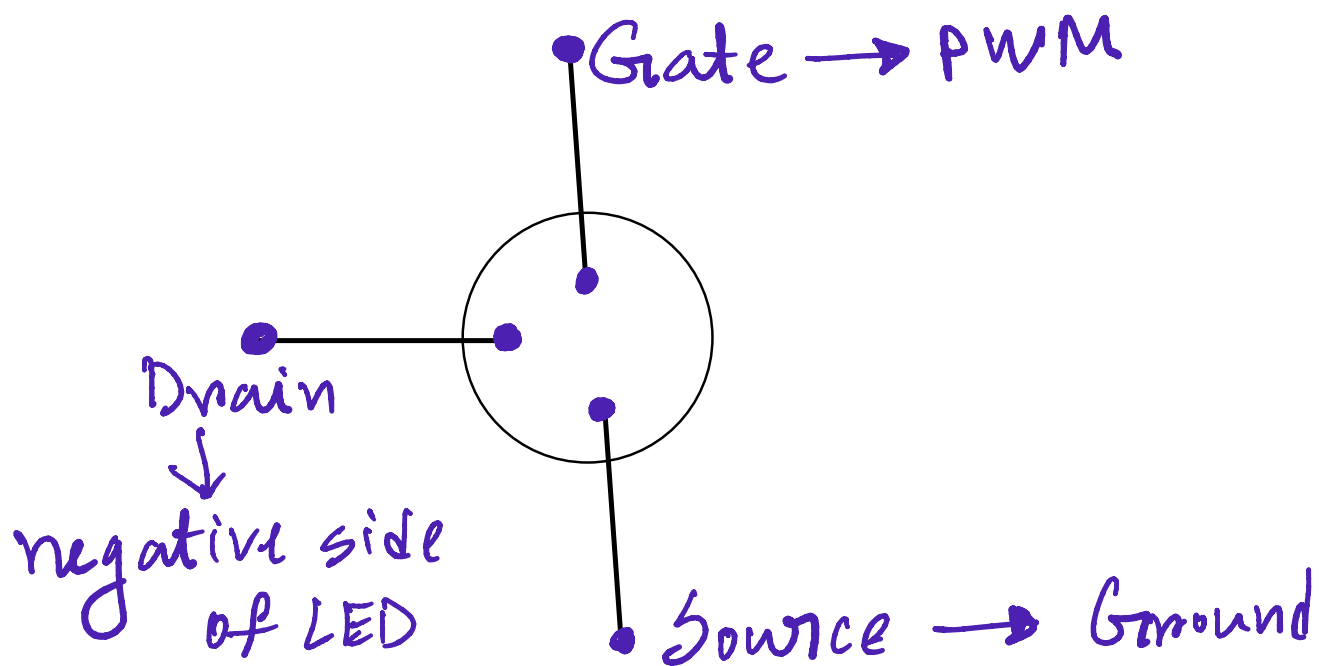
<http://www.youtube.com/user/greatscottlab>



* 555 timer chip →
i) alternative for generating PWM

ii) Allowing hobbyists to control duty cycle without complex setups
iii) can be wired by simply for various project

* Use MOSFET for powering / want to use a higher voltage



Video-3: Programming an Attiny + Homemade Arduino Shield

Components:

- i) WS2801 led strip ii) Attiny 85 iii) Atmega328p

i) WS2801 LED Strip

Work:

The WS2801 LED strip works as an individually addressable RGB lighting system.

Key Features:

- **Individually addressable** LEDs, meaning each LED can be controlled independently.
- **Uses SPI** (with Data and Clock lines) for communication.
- **Requires a microcontroller** (like Arduino, Raspberry Pi, ESP32, etc.) to control.
- **Power requirement:** Typically 5V, with higher current draw depending on the number of LEDs and brightness.

What It Can Do (It Works As):

- **Dynamic RGB Lighting** (you can set any LED to any color).
- **Animations** like fading, chasing, pulsing, rainbow effects.
- **Interactive Systems** (with sensors or music input).
- **Visual Displays** (text, patterns, waveforms).
- **Custom Lighting Scenes** programmable by you.

ii) Attiny 85 :

The ATtiny85 is a small but powerful microcontroller that can work as a controller for WS2801 LED strips.

Work as :



How the ATtiny85 Works in This Role:

The **ATtiny85** can:

- Generate **SPI signals** to control WS2801 LEDs.
- Be programmed with **Arduino code** (via the Arduino IDE).
- Run standalone without a full Arduino board once programmed.

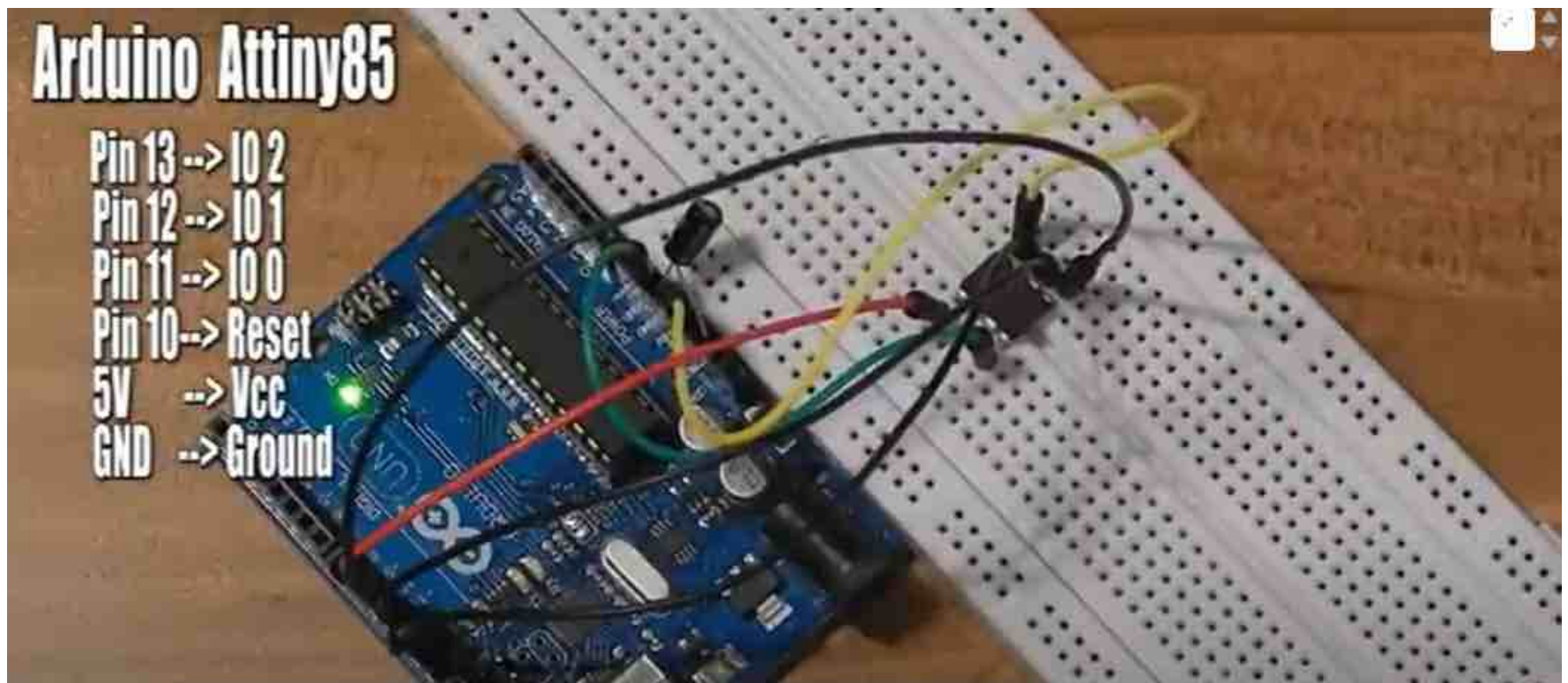
iii) Atmega 328p:

The **ATmega328P** (the microcontroller used in Arduino Uno) can **easily work as a controller for WS2801 LED strips**. It's more powerful than the ATtiny85, offering more memory, I/O pins, and flexibility.

ATmega328P + WS2801: How It Works

- You program the ATmega328P (usually via Arduino IDE).
- It sends **SPI data** (Data and Clock) to the WS2801 strip to control colors and patterns.
- This setup is great for DIY light projects, displays, or effects.

Arduino Setup:



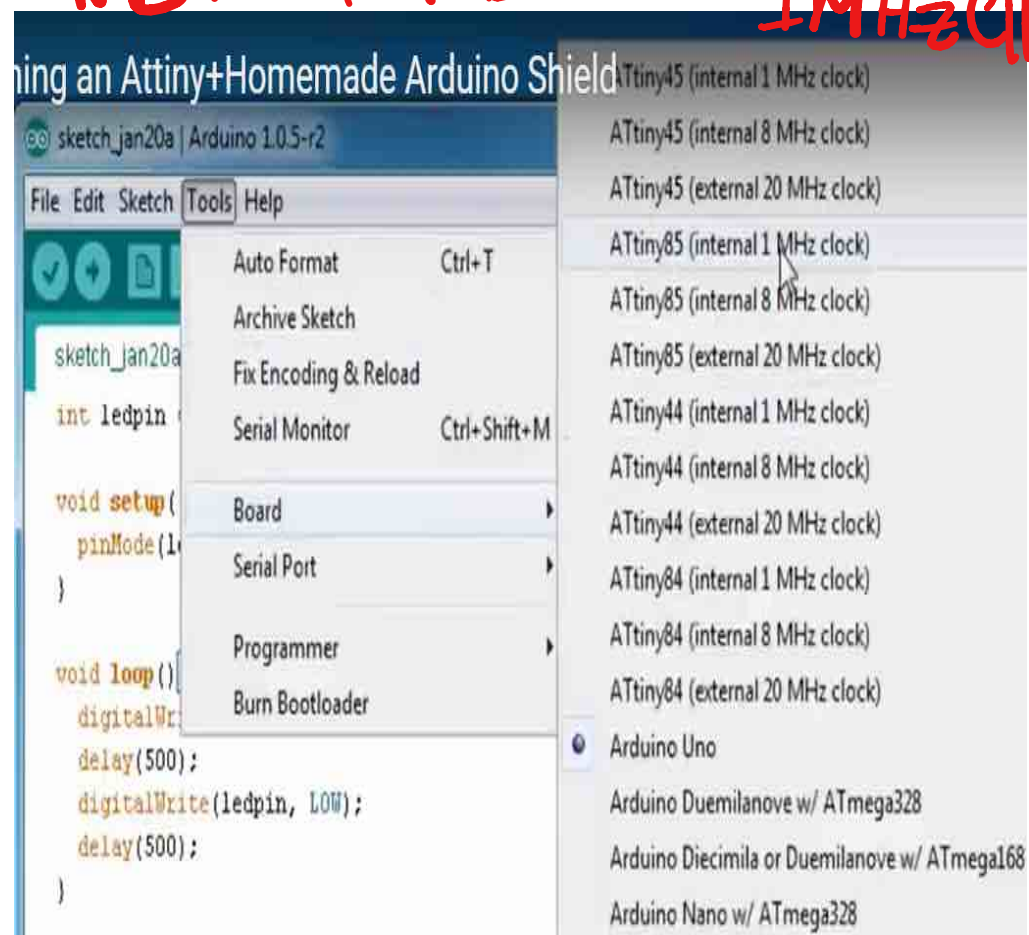
N.B. Don't forget to put a 10 microfarad capacitor betⁿ the reset pin of the Arduino and ground

NB: Select Brd → Attiny85 1MHz clk

```
File Edit Sketch Tools Help
sketch_jan20a $
int ledpin =3;

void setup() {
  pinMode(ledpin, OUTPUT);
}

void loop() {
  digitalWrite(ledpin, HIGH);
  delay(500);
  digitalWrite(ledpin, LOW);
  delay(500);
}
```



Summary:

Arduino software and an Arduino Uno as a programmer, and also demonstrates the creation of a simple homemade Arduino shield for easier ATtiny85 programming. Here's a detailed breakdown of the topics covered:

- **Introduction to the Project:** The video starts with the creator's goal of using a WS2811 LED strip with different animations controlled by a push button, requiring a separate microcontroller from an LED matrix project [[00:00](#)].
 - **Choosing the Microcontroller:** The creator explains why the ATtiny85 is a more cost-effective and suitable option compared to the ATmega328 (found on the Arduino Uno) for this specific task due to its smaller size, lower cost (around \$1), sufficient I/O pins (5), and flash memory (8 kilobytes) [[00:27](#)].
 - **Software Setup - Installing Arduino Software:** The first step involves downloading and installing an older version of the Arduino software (version 1.0.5), as the creator mentions that it did not work with the newer 1.55 beta version [[01:14](#)].
 - **Software Setup - Downloading and Installing ATtiny Board Data:** The tutorial then guides viewers on how to download the board data for the ATtiny85 from a provided link (highlowtech.org) [[01:31](#)]. This involves extracting the downloaded zip archive and copying the "attiny" folder into the "Arduino/hardware/arduino/" directory [[01:45](#)]. After restarting the Arduino software, new ATtiny boards should be visible [[01:53](#)].
 - **Preparing the Arduino Uno as an ISP Programmer:** The next step involves uploading the "ArduinoISP" sketch to the ATmega328 on the Arduino Uno. This sketch can be found in the examples section of the Arduino software [[02:04](#)].
 - **Understanding the ATtiny85 Pinout:** The video provides a clear overview of the ATtiny85's pin configuration, including ground (pin 4), VCC (pin 8), the five I/O pins (pins 2, 3, 5, 6, & 7 with Arduino code numbering shown), analog input capabilities of IO 2, 3, & 4, PWM signal generation on IO 0 & 1, and the reset pin (pin 1) [[02:16](#)].
 - **Wiring the Arduino Uno to the ATtiny85:** The tutorial explains the simple wiring connections between the Arduino Uno and the ATtiny85 for programming: Arduino pin 13 to ATtiny I/O 2, pin 12 to I/O 1, pin 11 to I/O 0, pin 10 to ATtiny reset, 5V from Arduino to ATtiny VCC (pin 8), and ground from Arduino to ATtiny ground (pin 4). Importantly, it highlights the need for a 10 microfarad capacitor between the reset pin of the Arduino and ground [[02:48](#)].
-

Components:

- i) Bluetooth module
- ii) Arduino nano
- iii) Voltage divider (two resistors)
2k Ω & 4.7k Ω
- iv) Anode RGB LED
- v) S2 Terminal for Bluetooth (App)



🧠 What is Arduino Nano?

The **Arduino Nano** is a compact, breadboard-friendly microcontroller board based on the **ATmega328P** microcontroller (same as Arduino Uno). It's ideal for compact, portable projects due to its small size and full functionality.

🧩 Pinout and Functions

The Arduino Nano has **30 pins** in total. Here's a breakdown of what each group of pins does:

⚡ Power Pins

Pin	Function
VIN	External power input (7–12V recommended)
GND	Ground
5V	Regulated 5V output
3.3V	Regulated 3.3V output (limited current)
RESET	Connect to reset button or use to reset from an external source
REF	Reference voltage for analog inputs (rarely used)

🔗 Communication Pins

Protocol	Pins Used	Purpose
UART	D0 (RX), D1 (TX)	Serial communication (USB or Bluetooth)
I2C	A4 (SDA), A5 (SCL)	Communication with I2C devices (e.g., OLED, sensors)
SPI	D10 (SS), D11 (MOSI), D12 (MISO), D13 (SCK)	Communication with SPI devices (e.g., SD card, displays)

💡 Special Pins

- **D13**: Connected to on-board LED (great for testing)
- **RESET**: Useful if you need to restart the board from an external circuit

🔑 Arduino Nano – Quick Key Uses

1. 🧠 **Controller** – Main brain of projects
2. 📡 **Sensor Hub** – Reads sensors, logs data
3. 🧱 **PWM Output** – Controls motors, dims LEDs
4. ⚡ **ADC** – Converts analog input to digital
5. ⚙️ **Automation** – Controls relays, devices
6. 🔗 **Comm Bridge** – Bluetooth/Wi-Fi interface
7. 🗣️ **I2C/SPI** – Talks to displays, sensors
8. 📖 **Learning Tool** – Great for beginners
9. 📦 **Compact MCU** – Fits small projects

✅ ADC (Analog-to-Digital Converter)

⚡ Digital I/O Pins (D0 – D13)

Pin	Function
D0 (RX)	UART receive pin
D1 (TX)	UART transmit pin
D2 – D13	General purpose digital input/output
D3, D5, D6, D9, D10, D11	PWM output (analogWrite)

PWM (Pulse Width Modulation) pins simulate analog output using digital signal switching.

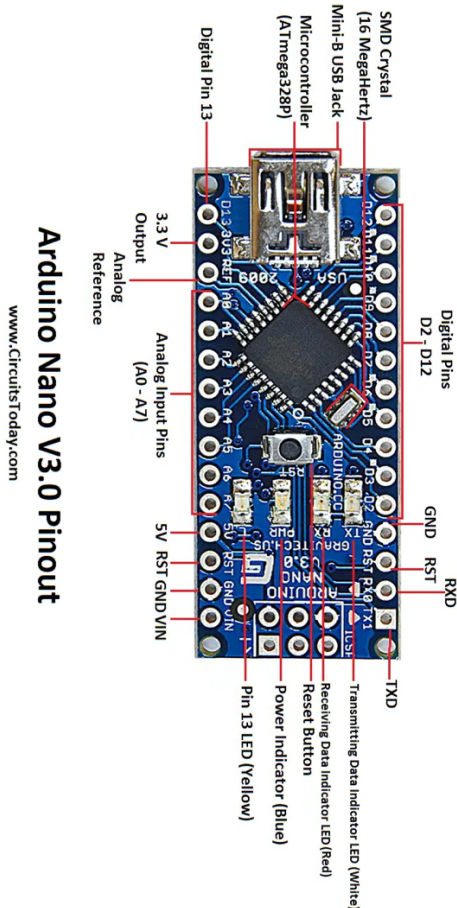
🔌 Analog Input Pins (A0 – A7)

Pin	Function
A0 – A5	Standard analog inputs (can also be used as digital I/O)
A6, A7	Analog input only , cannot be used as digital I/O

Each analog pin gives a **10-bit resolution** value (0–1023), useful for sensors.

📌 Notes

- Always **match voltage levels** of peripherals (Nano uses 5V logic).
- Avoid drawing too much current from **3.3V pin** — it's limited.
- Use **resistors** and **voltage dividers** when needed (e.g., for Bluetooth RX).



Video Summary:

The YouTube video explains how to use a Bluetooth module with an Arduino and an Android phone to control electronic components, like an RGB LED. Here's a summary:

- **Introduction:** The video demonstrates controlling gadgets with a smartphone via Bluetooth [\[00:00\]](#).
- **Hardware:** It uses a four-pin Bluetooth module and an Arduino Nano [\[00:07, 01:10\]](#).
- **Project Example:** The creator controls LEDs with it [\[00:16\]](#).
- **Bluetooth Module Purchase:** These are available on Amazon or eBay [\[00:50\]](#).
- **Voltage Divider:** Necessary because the Bluetooth module uses 3.3V, while the Arduino uses 5V [\[01:31\]](#). The video explains how to build one [\[02:19\]](#).
- **Wiring:** The video briefly describes wiring an RGB LED and the Bluetooth module to the Arduino [\[02:40\]](#).
- **Android App:** The creator recommends the "s2 terminal" app [\[03:13\]](#).
- **Arduino Code:** Code is available and needs modification for specific actions [\[03:28\]](#).
- **Uploading Code:** Disconnect the Bluetooth module before uploading [\[03:54\]](#).
- **Bluetooth Pairing:** The module's name is "hc-05," and common pairing codes are 1234 or 0000 [\[04:01\]](#).
- **Functionality:** Sending code words via the app controls the RGB LED [\[04:16\]](#).